K8S-03: GitOps for DynaKube

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 3 of 14 | **Created:** January 2026 | **Last Updated:** 11/03/2026

Managing DynaKube with ArgoCD and Flux

GitOps enables declarative, version-controlled management of your Dynatrace monitoring configuration. This notebook covers integrating DynaKube with popular GitOps tools: ArgoCD and Flux.

Table of Contents

1. GitOps Principles for Observability
2. ArgoCD Integration
3. Flux Integration
4. Secret Management
5. Multi-Cluster Patterns
6. Progressive Rollouts
7. Troubleshooting GitOps Deployments

Prerequisites

Requirement	Details
GitOps Tool	ArgoCD v2.x or Flux v2.x
Git Repository	Write access for configuration

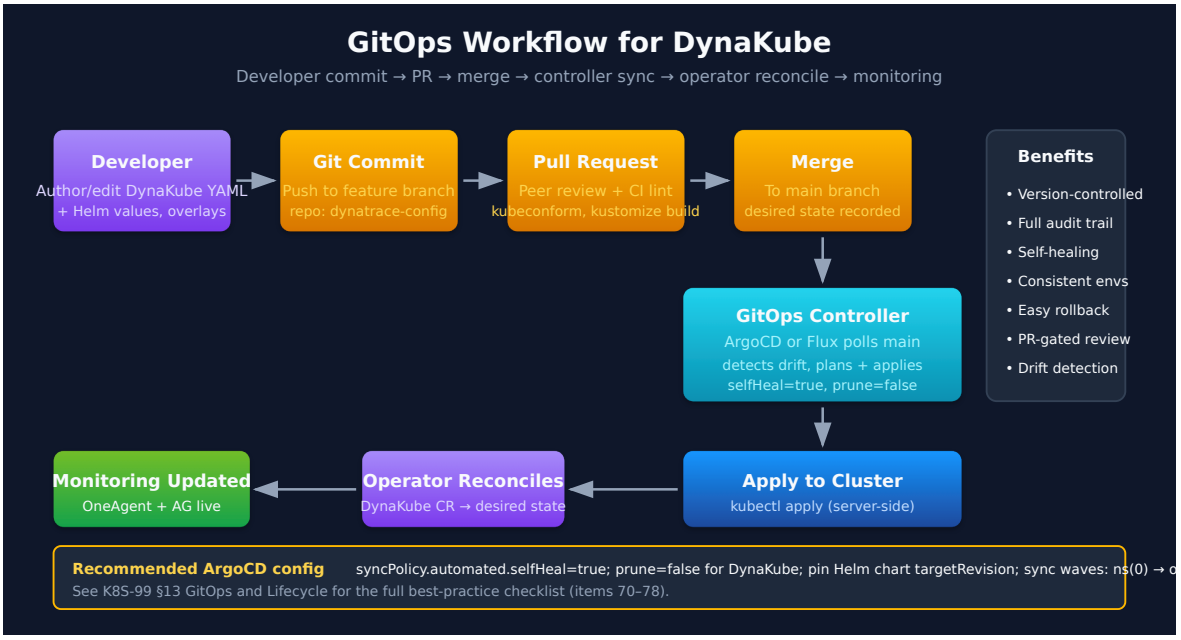
Requirement	Details
Kubernetes	Dynatrace Operator installed
Knowledge	Familiarity with K8S-02: DynaKube Deployment

1. GitOps Principles for Observability

Why GitOps for Monitoring?

Benefit	Description
Version Control	Track all config changes in Git history
Audit Trail	Who changed what, when, and why
Consistency	Same config across dev/staging/prod
Self-Healing	Automatic drift correction
Rollback	Revert by reverting Git commits

GitOps Workflow for DynaKube



Repository Structure

```
monitoring-config/
├── base/
│   ├── namespace.yaml
│   ├── operator/
│   │   └── kustomization.yaml
│   └── dynakube/
│       ├── dynakube.yaml
│       └── kustomization.yaml
├── overlays/
│   ├── dev/
│   │   ├── kustomization.yaml
│   │   └── dynakube-patch.yaml
│   ├── staging/
│   │   └── ...
│   └── prod/
│       └── ...
└── clusters/
    ├── dev-cluster/
    ├── staging-cluster/
    └── prod-cluster/
```

2. ArgoCD Integration

ArgoCD Application for Operator

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace-operator
  namespace: argocd
spec:
  project: monitoring
  source:
    repoURL: https://raw.githubusercontent.com/Dynatrace/dynatrace-
operator/main/config/helm/repos/stable
    chart: dynatrace-operator
    targetRevision: 1.10.4  # Pin to a version you have validated — check
https://github.com/Dynatrace/dynatrace-operator/releases for latest
    helm:
      releaseName: dynatrace-operator
      values: |
        platform: kubernetes
        csidriver:
          enabled: true
  destination:
    server: https://kubernetes.default.svc
    namespace: dynatrace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

> **Choosing the pinned version.** A GitOps pin is copy-pasted far more often than it is revisited, so pick it deliberately:
>
> | Constraint | Why |
> | --- | --- |
> | **Do not pin below 1.4.1** | Operator 1.3.0 through 1.4.0 are inside the CSI driver liveness-probe crash-loop window; 1.4.1 adjusted the probe parameters. See K8S-09 §2. |

Constraint	Why
Skip 1.10.0	Its own release notes advise skipping it (auto-update defect), and it is now flagged `prerelease: true` on the GitHub releases page — a signal you can verify yourself before pinning. Fixed in 1.10.1.
1.11.1 is the newest release (10/22/2026)	A patch on 1.11.0. Operator 1.11.1 (DT docs): *"Fixed a security vulnerability (CVE-2026-111943) by updating the Go version used to build Dynatrace Operator to 1.27.3."* 1.11.0 (10/01/2026) removes DynaKube `v1beta4` from the CRD — a manifest still declaring it fails to apply after the upgrade — and adds image-volume injection and `spec.kubernetesMonitoring`. Move every DynaKube manifest in Git to `v1beta6` *before* bumping this pin, and read the notes of each minor version you cross (K8S-02 §8).
1.10.4 is the validated pin in this series	Released October 22, 2026. **1.10.3** (October 15, 2026) fixed the CSI driver cleanup that could delete OneAgent code modules still in use by running pods (K8S-09 §2; the Helm chart enables the CSI driver by default), and **1.10.4** fixes CVE-2026-111943 — its release notes say *"We recommend that you upgrade to this version."* 1.10.4 (DT docs) is *"released only on GitHub (Helm and manifests)"*; OperatorHub and cloud-marketplace installs get the same fix in 1.11.1. It also carries the 1.10.2 fixes (July 30, 2026): a workload/namespace **tagging-precedence regression** (subsequent matching rules used to overwrite earlier ones; now only the *first* matching rule per key applies — a behaviour change, see K8S-10), a `dynatrace-webhook` `CrashLoopBackOff` under the gVisor runtime class, injected pods hanging on the OneAgent-binary download (timeout raised to 15 minutes), and it now logs metadata-enrichment rules it cannot apply instead of dropping them silently.
Earlier 1.10.x pins keep running	Estates move GitOps pins on their own cadence. 1.10.1 and 1.10.2 still fix the 1.10.0 defects, but lack the CSI cleanup and CVE fixes; 1.10.1 is also subject to the OpenShift-manifest caveat in K8S-09 §2. Read the K8S-10 tagging note before bumping a cluster whose enrichment relies on last-rule-wins ordering.
Below 1.6.0 cannot upgrade directly	Operator 1.11.0 (DT docs): *"Minimum operator version required for direct upgrade: 1.6.0"* — older clusters step through intermediate versions.

Pin an exact version rather than a range for the operator itself — an operator upgrade can change chart *defaults* (see K8S-02 §8), and a range lets that happen

> without a Git commit to review.

ArgoCD Application for DynaKube

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynakube
  namespace: argocd
spec:
  project: monitoring
  source:
    repoURL: https://github.com/your-org/monitoring-config
    targetRevision: main
    path: overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: dynatrace
  syncPolicy:
    automated:
      prune: false  # Don't auto-delete DynaKube
      selfHeal: true
```

ArgoCD App of Apps Pattern

Manage multiple clusters from a single Application:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: monitoring-apps
  namespace: argocd
spec:
  project: monitoring
  source:
    repoURL: https://github.com/your-org/monitoring-config
    targetRevision: main
    path: clusters
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

3. Flux Integration

Flux HelmRelease for Operator

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: dynatrace
  namespace: flux-system
spec:
  interval: 1h
  url: https://raw.githubusercontent.com/Dynatrace/dynatrace-
operator/main/config/helm/repos/stable
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: dynatrace-operator
  namespace: dynatrace
spec:
  interval: 5m
  chart:
    spec:
      chart: dynatrace-operator
      version: "1.10.4"  # Pin exactly — see the version-choice table in §2
(do not pin below 1.4.1; skip 1.10.0)
      sourceRef:
        kind: HelmRepository
        name: dynatrace
        namespace: flux-system
  values:
    platform: kubernetes
    csidriver:
      enabled: true
```

> **API versions.** Current Flux controllers serve `HelmRelease` only at `helm.toolkit.fluxcd.io/v2` and `HelmRepository` only at `source.toolkit.fluxcd.io/v1`; the older `v2beta1` / `v1beta2` manifests fail to apply there. CRD versions read from the `fluxcd/helm-controller` and `fluxcd/source-controller` repositories, 10/02/2026.

Flux Kustomization for DynaKube

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynakube
  namespace: flux-system
spec:
  interval: 10m
  path: ./overlays/prod
  prune: true
  sourceRef:
    kind: GitRepository
    name: monitoring-config
  healthChecks:
    - apiVersion: dynatrace.com/v1beta6
      kind: DynaKube
      name: dynakube
      namespace: dynatrace
  dependsOn:
    - name: dynatrace-operator
```

Flux GitRepository Source

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: monitoring-config
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/your-org/monitoring-config
  ref:
    branch: main
  secretRef:
    name: git-credentials
```

4. Secret Management

API tokens should never be stored in Git. Use these approaches:

Option 1: External Secrets Operator

```
apiVersion: external-secrets.io/v1   # v1beta1 is no longer served by current
ESO releases
kind: ExternalSecret
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: ClusterSecretStore
    name: vault  # or aws-secrets-manager, gcp-secret-manager
  target:
    name: dynakube
    creationPolicy: Owner
  data:
    - secretKey: apiToken
      remoteRef:
        key: dynatrace/operator-token
    - secretKey: dataIngestToken
      remoteRef:
        key: dynatrace/data-ingest-token
```

Option 2: Sealed Secrets

```
# Create sealed secret from raw secret
kubectl create secret generic dynakube \
  --namespace dynatrace \
  --from-literal=apiToken=&lt;your-api-token&gt; \
  --from-literal=dataIngestToken=&lt;your-data-ingest-token&gt; \
  --dry-run=client -o yaml | \
  kubeseal --format yaml &gt; dynakube-sealed.yaml
```

```
# Sealed secret (safe to commit)
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  encryptedData:
    apiToken: AgBy8hM...
    dataIngestToken: AgCtr9...
```

Option 3: SOPS with Age/GPG

```
# .sops.yaml in repo root
creation_rules:
  - path_regex: .*.secret.yaml$
    age: age1ql3z7hjy54pw...
```

```
# Encrypt secrets
sops --encrypt secrets.yaml &gt; secrets.secret.yaml

# Flux decrypts automatically with kustomize-controller
```

5. Multi-Cluster Patterns

Kustomize Overlays by Environment

Base DynaKube (`base/dynakube/dynakube.yaml`):

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
spec:
  apiUrl: PLACEHOLDER  # Patched per environment
  oneAgent:
    cloudNativeFullStack: {}
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
```

Production Overlay (`overlays/prod/kustomization.yaml`):

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: dynatrace
resources:
  - ../../base/dynakube
patches:
  - patch: |-
      - op: replace
        path: /spec/apiUrl
        value: https://your-tenant.live.dynatrace.com/api
    target:
      kind: DynaKube
      name: dynakube
```

Environment-Specific Configuration

Environment	Config Difference
Dev	Lower resources, fewer replicas
Staging	Match prod config, different tenant
Prod	High availability, more resources

Dev Patch (`overlays/dev/dynakube-patch.yaml`):

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
spec:
  activeGate:
    replicas: 1
    resources:
      requests:
        cpu: 100m
        memory: 256Mi
```

6. Progressive Rollouts

ArgoCD Sync Waves

Control deployment order:

```
# Wave 0: CRDs and Namespace
apiVersion: v1
kind: Namespace
metadata:
  name: dynatrace
  annotations:
    argocd.argoproj.io/sync-wave: "0"
---
# Wave 1: Operator
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace-operator
  annotations:
    argocd.argoproj.io/sync-wave: "1"
---
# Wave 2: DynaKube
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynakube
  annotations:
    argocd.argoproj.io/sync-wave: "2"
```

Flux Dependency Ordering

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynakube
spec:
  dependsOn:
    - name: dynatrace-operator
    - name: external-secrets  # If using ESO for tokens
```

Canary Deployments

Roll out monitoring changes gradually:

1. Deploy to dev cluster → validate
2. Deploy to staging → run tests
3. Deploy to prod-canary (subset) → monitor
4. Deploy to prod (all clusters)

7. Troubleshooting GitOps Deployments

ArgoCD Troubleshooting

```
# Check application status
argocd app get dynakube

# View sync diff
argocd app diff dynakube

# Force sync
argocd app sync dynakube --force

# View logs
kubectl -n argocd logs -l app.kubernetes.io/name=argocd-application-controller
```

Flux Troubleshooting

```
# Check kustomization status
flux get kustomizations

# Check helm releases
flux get helmreleases -n dynatrace

# Force reconciliation
flux reconcile kustomization dynakube --with-source

# View events
kubectl -n flux-system get events --sort-by='.lastTimestamp'
```

Common Issues

Issue	Cause	Solution
Secret not found	External secret sync delay	Check ESO status, wait for sync
CRD not found	Operator not deployed	Check operator app status
Sync failed	YAML syntax error	Validate with `kubectl --dry-run`
Drift detected	Manual changes	Revert or accept drift in Git

```
// GitOps controller activity, from Kubernetes events (events, not logs — K8S-01 §6).
// Shows sync-driven pod churn and failures in the controller namespaces.
fetch events, from:-24h
| filter event.provider == "KUBERNETES_EVENT"
| filter in(k8s.namespace.name, {"argocd", "flux-system"})
| fields timestamp, k8s.namespace.name,
dt.kubernetes.event.involved_object.kind,
    dt.kubernetes.event.involved_object.name, dt.kubernetes.event.reason,
dt.kubernetes.event.message
| sort timestamp desc
| limit 30
```

```
// What changed in the dynatrace namespace after a DynaKube sync?
// A DynaKube change rolls OneAgent / ActiveGate / webhook pods; their
Kubernetes events
// (Killing, BackOff, FailedScheduling, Unhealthy …) show whether the rollout
settled.
fetch events, from:-24h
| filter event.provider == "KUBERNETES_EVENT" and k8s.namespace.name ==
"dynatrace"
| summarize {events = countDistinctExact(event.id), last_seen =
max(timestamp)},
    by:{dt.kubernetes.event.involved_object.kind, dt.kubernetes.event.reason}
| sort last_seen desc
| limit 20
```

Next Steps

With GitOps configured, proceed to:

Next Notebook	Topic
K8S-04: Cluster Health Monitoring	Deep-dive into cluster metrics
K8S-05: Workload Monitoring	Application-level observability
K8S-06: Namespace Organization	Boundaries and access control

Summary

In this notebook, you learned:

- GitOps principles and benefits for monitoring
- ArgoCD Application definitions for operator and DynaKube
- Flux HelmRelease and Kustomization for DynaKube
- Secret management with External Secrets, Sealed Secrets, and SOPS
- Multi-cluster patterns with Kustomize overlays
- Progressive rollout strategies
- Troubleshooting GitOps deployments

References

- Set up Dynatrace on Kubernetes (DT docs)
- DynaKube parameters (DT docs)
- Dynatrace Operator releases (Dynatrace GitHub)
- Dynatrace Operator Helm chart (Dynatrace GitHub)
- ArgoCD documentation (argoproj.io)
- Flux documentation (fluxcd.io)
- External Secrets Operator (external-secrets.io)
- Kustomize (kustomize.io)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.